Interim Report: The Automation Auditor

Name: Chalie Lijalem

Architecture Decisions

- **Pydantic Models over Dicts**: We use Pydantic for all structured data (Evidence, JudicialOpinion, etc.) to ensure strict type validation, serialization, and IDE support. This prevents subtle bugs and enforces schema consistency across all nodes and state transitions, which is critical for a multi-agent system.

- **AST Parsing for Code Analysis**: Instead of regex, we use Python's built-in `ast` module to analyze repository code. This allows us to reliably detect `StateGraph` instantiations and method calls like `add_edge` and `add_node`, ensuring robust and maintainable code analysis that is resilient to code formatting and comments.

- **Sandboxing Strategy**: All repository cloning and analysis is performed inside a `tempfile.TemporaryDirectory()` context. This ensures that each analysis runs in a secure, isolated environment, with automatic cleanup and no risk of polluting the host filesystem. All subprocesses (e.g., `git clone`, `git log`) are run with explicit error handling.

Known Gaps & Plan for Judicial Layer

- **Current Gaps**:
  - The judicial layer (Prosecutor, Defense, Tech Lead) is not yet integrated in the interim graph.
  - No synthesis engine (Chief Justice) to aggregate and resolve conflicting opinions.
  - No scoring or rubric-based aggregation in the final report.
- **Concrete Plan**:
  1. **Implement Judicial Nodes**: Each judge will use LangChain's `.with_structured_output(JudicialOpinion)` to ensure strictly typed outputs. System prompts will be tailored for each role (critical, supportive, architectural).
     - *Risks/Failure Modes*: LLMs may not always return strictly typed outputs, leading to parsing errors. Persona prompts may not elicit sufficiently distinct reasoning. Mitigation: Add output validation and fallback logic; iterate on prompt engineering.
  2. **Synthesis Engine**: The Chief Justice node will analyze the variance in scores and arguments, resolve conflicts, and generate a final `AuditReport` using a structured LLM call.
     - *Risks/Failure Modes*: High variance in judge scores may not be resolved cleanly, or the synthesis may fail to capture nuanced disagreements. Mitigation: Explicitly surface score variance and

require the LLM to address controversy in the summary; add manual review option.

3. **Graph Wiring & Sequencing**: The graph will be extended to include parallel fan-out to judges after evidence aggregation, followed by a synthesis fan-in to the Chief Justice node.
    - *Sequencing/Prioritization*: Priority will be given to integrating the judicial layer first, followed by synthesis logic. Testing will focus on edge cases (conflicting opinions, missing evidence). The rubric-driven aggregation will be implemented last, once the core dialectical flow is validated.

Planned StateGraph Flow

Below is diagram of the planned StateGraph architecture for the full system. Edge labels show the primary pieces of state carried between nodes and conditional/error paths are shown explicitly.

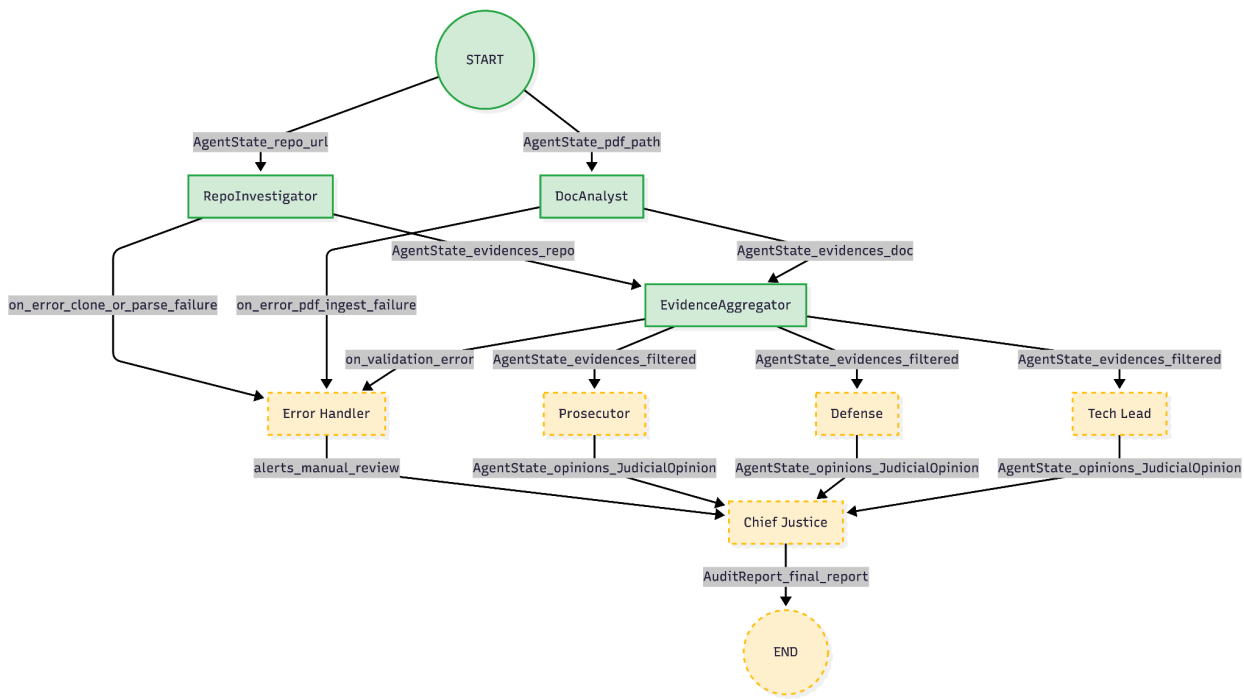

Diagram notes

- Edge labels explain the primary data being passed (for example `AgentState.evidences` holds categorized lists of `Evidence` objects; `AgentState.opinions` holds `JudicialOpinion` objects produced by each judge).
- Conditional/error edges (`--|on error: ...|`) explicitly capture failure modes such as clone failures, PDF ingest errors, or validation problems encountered during evidence aggregation. These routes terminate in an `Error Handler` (which raises alerts and can escalate to manual review or routing to the `ChiefJustice`).

- The `EvidenceAggregator` is a synchronization/fan-in node: it receives evidence from parallel detectives, validates/filters it, and then fans it out to judicial nodes. Labels show whether the evidence is `repo` or `doc` scoped.
- The `ChiefJustice` expects typed inputs (`JudicialOpinion` objects) and outputs a typed `AuditReport` as `final_report`.

Summary

The Automation Auditor interim submission demonstrates a robust, modular foundation for automated code and documentation analysis. The next phase will focus on dialectical reasoning, structured opinion synthesis, and rubric-driven reporting.